

Magus Match

An Endless Match-3 Save-the-Hero Adventure

Field	Value
Category	H4 — Hybrid-Casual Puzzle & Logic (Match-3 + Save the Hero)
Batch	Snackable Games Batch 2
Target Platform	Meta Horizon Worlds (ported from Phaser 3 / TypeScript prototype)
Orientation	Vertical (portrait) mobile
Primary Audience	Young Adults / Adults
Run Length	30 seconds – 5 minutes per run, 1–10+ levels per run
Time to First Fun	< 6 seconds
Distribution Channel	Instagram Reels (playable ad creative)
Author	Frank
Doc Version	v2.0 — Magus Match retheme

1. High Concept

The one-sentence pitch

You are a heroic mage-knight chasing a kidnapped prince through randomly generated Journey and Trial levels — match tiles to forge paths and cast spells, save the prince, watch him get abducted again, repeat until your three lives run out.

Why this works as a Snackable Game

Magus Match takes the proven "Save the Hero" match-3 hook and reframes it as an endless run instead of a hand-designed campaign. Two level types alternate randomly: Journey (path-clearing, turn-based) and Trial (real-time wave defense). Each level cleared rescues the prince — but a smash-cut animation reveals the prince getting yanked off-screen by an unseen force, and the next level begins. The running gag is the meta. Players play "one more level" because each rescue is also a setup.

The endless structure plus a global high-score leaderboard turns a snackable match-3 into a session game. A run lasts 1–10+ levels depending on player skill; a session lasts as long as the player wants. The Reels hook is the rescue-then-re-kidnapped gag — it's funny, it's repeatable, and it's a narrative engine that explains why the game never ends.

Viral hook (describable in one sentence)

The hook

"Save the prince. Then save him again. Then save him again, because he keeps getting taken." — viewers see the running gag in 6 seconds and immediately understand both the loop and the punchline.

Design pillars

- **Two loops, one hero.** Journey levels test spatial planning under move budget. Trial levels test reflexes under real-time pressure. Alternating them keeps a session feeling varied without adding a third system.
- **Endless, randomized, leaderboarded.** No hand-designed campaign. Levels are procedurally generated; difficulty rises steadily; a 3-life run produces a final score and a leaderboard rank.
- **The running gag IS the meta.** Every level ends with the prince celebrating, then being yanked off-screen mid-cheer. There is no progress map, no castle decoration, no character upgrade tree. The unseen abductor IS the meta-progression — the player wonders who or what keeps taking the prince. We never tell them.

- **Hand-rigged Blender puppets, AI-generated everything else.** Characters use 2D cutout animation in Blender (per Xan's pipeline); environments and tiles are AI-generated 2D art. This hybrid keeps character motion expressive while keeping environment art volume cheap.
- **Shareable endings.** Each run-end screen with high-score and leaderboard is built for screenshot capture; each level's rescue-then-yanked beat is a 3-second loop perfect for Reels.

2. Core Gameplay Loop

2.1 Run Structure

A "run" is the player's session from press-start to game-over. Runs have no fixed length. The structure:

1. Player taps Play. They begin Level 1 with 3 lives.
2. A level type is randomly chosen (50/50 Journey or Trial, with a guarantee that the first level of any run is a Journey to ease the player in).
3. The player attempts the level. Success = level cleared, points awarded, prince celebrates and gets yanked off-screen. Failure = lose 1 life.
4. If lives > 0, advance to the next level. Difficulty scales steadily.
5. If lives = 0, the run ends. Show end-of-run screen with final score, high score, leaderboard top 10, and Try Again.

2.2 Screen Layout

Both level types use the same fixed vertical screen layout, divided into three sections. All dimensions scale from a 1920px base screen height (and full screen width):

Section	Height (base 1920)	Contents
Hero Stage (3D)	600 px, full width, top-aligned	The 3D scene. Mage-knight protagonist, prince in his cage, monsters in Trial levels, path goal markers in Journey levels. Animated via Blender-rigged 2D cutout puppets composited into the 3D stage.
Gameplay UI	200 px, full width, sandwiched	The HUD strip. Level number, lives remaining, score, current objective indicator (move budget for Journey, wave progress for Trial), and any state-specific banners or notifications.
Match Board	1120 px, full width, bottom-aligned	The 8×8 board lives here. The board itself is square and scales to fit the full width of the section. The board is centered horizontally and vertically within this 1120 px section, leaving symmetric vertical margin above and below the board for visual breathing room.

All three sections scale proportionally on devices with non-1920 vertical resolution; the ratios (600 / 200 / 1120) are preserved. The split is consistent across both level types so the player always knows where their attention belongs: the 3D action happens up top, the persistent HUD is in the middle, and all interactive control happens at the bottom on the board.

Board sizing within the Match Board section

On a 1080×1920 reference device, the Match Board section is 1080 wide × 1120 tall. The board is square and scales to fit the section's full width — so the board itself is 1080×1080. That leaves 40 px of vertical margin total (20 px above, 20 px below the board) within the Match Board section. On wider/narrower devices the board's edge length tracks screen width, with the section height tracking the 1120/1920 ratio of total screen height.

2.3 The Two Level Types

Journey Levels — Path the Mage Through

Turn-based. The player has a fixed move budget. The 8×8 board is overlaid with a special "land" tile type (e.g., grass, stone, sand) seeded into specific cells. The mage-knight stands at one edge of the board (top-left); a goal flag stands at the opposite edge (bottom-right). Each match of 3+ land tiles converts those cells into permanent navigable land, and the mage walks one step along the longest contiguous land path on his turn. Win condition: the mage reaches the goal flag. Fail condition: move budget exhausted before reaching the flag.

The land doesn't disappear — it persists. So matches are about painting a path, not just clearing tiles. Standard non-land matches still resolve (and grant combo points) but don't advance the mage.

Trial Levels — Defeat the Wave

Real-time. A wave of monsters (kobolds, taller kobolds, mini-bosses at higher difficulty) spawns at the top of the Hero Stage and walks toward the mage at the bottom of the stage. The match board is live — the player can swipe at any time. Each match casts a spell: a magical projectile fires from the board and instantly hits the nearest monster. Match color determines the spell's visual style (Fire / Ice / Lightning / Earth) but all four spells deal identical instant single-target damage; the schools are mechanically interchangeable, visually distinct (see §3.4). Win condition: all monsters defeated. Fail condition: any monster reaches the mage.

This is the more frantic, twitchy mode. Matching speed (matches per second) is itself a scoring axis. The board cascades and refills aggressively to keep tempo high.

2.4 The Core Loop Diagram

Per-level loop

ENTER LEVEL → randomly assigned Journey or Trial → MATCH TILES (turn-based or real-time) → progress objective (path or wave) → WIN (prince celebrates, yanked off-screen) or FAIL (lose a life) → NEXT LEVEL or GAME OVER.

Per-run loop

PLAY → clear levels, lose lives → 3 lives lost → see final score + leaderboard → TRY AGAIN.

2.5 Win & Fail States

Win — "Hero Saves the Prince!"

Both level types resolve into the same victory beat: the mage strikes a victory pose, the prince's cage is shown in close-up, the prince cheers and pumps his fists — and then a giant, off-screen unseen force snatches the cage upward and out of frame in a cartoon "WHOOSH." The mage looks up, blinks, shrugs, and the next level begins. This animation is ~2.5 seconds, designed to loop cleanly. It is the running gag and the soul of the game.

Fail — Journey

Move budget hits 0 with the path incomplete. The mage scratches his head, the goal flag droops sadly, the prince's cage trembles. The mage loses a life (heart icon shatters in the HUD). Brief 1.5s beat, then transition to next level (or end-of-run screen).

Fail — Trial

A monster reaches the mage. The mage staggers backward, comically dazed (stars circling his head). The cage trembles. The mage loses a life. Brief 1.5s beat, then transition.

Why both outcomes are designed for Reels

- The win-then-yanked gag is the single best Reels asset in the game — universally legible, comically timed, infinitely repeatable.
- Each rescue plays against a different backdrop biome and a different abductor visual hint, so consecutive wins produce visually distinct clips for ad cutting.
- Fail beats are deliberately gentle — the mage is bonked, not killed — keeping the tone family-friendly.
- End-of-run screen with leaderboard rank is a screenshot moment for high-scoring players.

3. Mechanics

3.1 Match-3 Foundation

Standard swipe-to-swap mechanics, deliberately tuned to feel faster than Candy Crush and match Royal Match's "fastest switcher" feel. Four tile types representing the four spell schools (Fire, Ice, Lightning, Earth) — see §3.7 — chosen because four types is the sweet spot for reading the board on a small portrait screen, plus a special LAND tile type that appears only in Journey levels.

Rule	Behavior
Swap	Tap-and-drag a tile one square in any cardinal direction. If the swap creates a match of 3+, it resolves. If not, tiles snap back — no life lost.
Match	3 in a row = clear. 4 in a row = creates a Rocket (clears a row or column). 5 in an L/T = creates a TNT (clears 3×3). 5 in a line = creates a Lightball (clears all tiles of one color).
Cascade	Tiles above fall down. The player can swipe during cascades (concurrent matching) — this is non-negotiable, it is what makes the game feel fast.
Shuffle	If no matches are possible, the board auto-shuffles instantly. No booster prompt, no modal. Removes a known friction point from traditional match-3.
Move budget	15 moves per level, extendable to 20 in later levels.

3.2 Journey Levels

Journey is the slower, more cerebral level type. It's also the easier of the two to learn, which is why it's guaranteed as the first level of every run.

Setup

The 8×8 board contains a mix of regular tiles (4 types — see §3.7) and "land" tiles. Land tiles look distinctly different — they're rendered as terrain chips (grass, stone, sand, water) rather than the magical-symbol regular tiles. The mage-knight stands on a starting cell at one corner of the board (top-left by default); a goal flag stands at the opposite corner (bottom-right).

Mechanic

6. Player swipes to swap tiles, just like standard match-3.
7. When 3+ land tiles match, those cells are converted to PERMANENT navigable land. They glow briefly and become walkable. Land tiles do not clear from the board — instead, they transform into a "path" overlay state on the cell.
8. After the player's move (and any cascades), the mage takes a turn: he walks one step along the longest contiguous path that connects to his current cell, choosing the cell closest to the goal flag if multiple options exist.

9. Standard non-land matches still clear normally and award combo points, but don't move the mage.
10. Move budget ticks down with every swap (matched or not, like classic Royal Match rules).

Win condition

The mage's foot reaches the cell containing the goal flag. Triggers victory cinematic — mage strikes pose, prince's cage shown, prince celebrates, prince yanked off-screen by unseen force.

Fail condition

Move budget reaches 0 with the mage not yet at the flag. The path is incomplete. Mage scratches head, cage trembles, lose 1 life.

Why Journey is Journey

This level type rewards spatial thinking — players want to match land tiles in a contiguous chain, not just anywhere. They're drawing a road across the board with each match. Standard match-3 instinct (match anywhere for points) is suboptimal here; the player must learn to plan land-tile placement. This is the depth that keeps experienced players engaged across long runs.

3.3 Trial Levels

Trial is the faster, more reflex-driven level type. It's where the mage-knight feels most like a hero.

Setup

The 8×8 board contains all standard tiles, no land tiles. The Hero Stage at the top of the screen shows the mage at the bottom-center; a horizontal spawn line at the top of the stage spawns monsters that walk downward toward the mage. Multiple monsters can be on-screen simultaneously, in waves.

Mechanic

11. There is no move budget. The board is always live.
12. Player matches tiles. Each match casts a spell — an instant projectile streaks from the matched tile cluster to the nearest enemy and damages it on the same frame.
13. Match color determines spell type (see §3.4).
14. Monsters take damage from spell hits. Each monster type has an HP value scaled by current run difficulty.
15. Monsters walk steadily downward. Their walking speed scales with run difficulty.
16. If a monster reaches the mage's standing line, the mage staggers, the level fails, lose 1 life.

Spell projectiles

Spell projectiles are instant: damage applies on the same frame the match resolves. The visual is a fast streak — projectile spawns at the matched cell, traces a quick line to the target, and impacts within ~80ms. Mechanically the hit registers immediately; the visual streak is for feedback, not a flight-time gating mechanic. On impact, the spell triggers a damage popup and a small VFX burst on the monster, themed to the spell school (flame burst / ice shatter / electric arc / dust burst).

Win condition

All monsters in the wave defeated before any reach the mage. Triggers victory cinematic — mage twirls his staff, prince celebrates, prince yanked off-screen.

Fail condition

Any monster touches the mage's standing line. Mage is bonked (cartoon stars), lose 1 life.

Wave composition

Each Trial level has a defined wave manifest — the number of monsters, their types, and their spawn timing. Lower-difficulty levels have small waves of basic kobolds; higher-difficulty levels add taller kobolds and eventually mini-bosses with multi-stage health bars. Wave generation is procedural — see §3.5.

3.4 Spell & Damage System

The four tile colors map to four spell schools. This system is dormant in Journey levels (where matches paint paths) but is the dominant mechanic in Trial levels. By design, all four spells are mechanically identical — instant projectile, instant single-target damage, no status effects. The schools differ in visuals and audio only. This keeps tuning simple, makes board state easy to read, and means players never feel they should have matched a different color.

Tile color	Spell school	Effect on enemies	Effect on Journey
Red	Fire	Instant projectile, instant single-target damage. Visual: crackling flame, fiery impact burst.	No special effect — just a regular match.
Blue	Ice	Instant projectile, instant single-target damage. Visual: icicle shard, frozen-shatter impact.	No special effect.
Gold	Lightning	Instant projectile, instant single-target damage. Visual: jagged bolt, electric-arc impact.	No special effect.
Green	Earth	Instant projectile, instant single-target damage. Visual: rock shard, dust-burst impact.	No special effect.

Damage scaling

Match size scales damage. A 3-match casts a basic spell. A 4-match casts a stronger spell with a slightly bigger projectile and roughly 2× damage. A 5-match (creating a power-up tile) is a special case — see power-up behavior below. The damage values are identical across all four schools at the same match size.

- Rocket (4-match line) → fires multiple instant projectiles at the row or column of enemies, hitting up to N targets in turn order. No piercing physics, no AoE — just sequential single-target hits.
- TNT (L/T 5-match) → fires several instant projectiles spread across the wave, each hitting one enemy. Functions as a multi-shot, NOT an AoE blast.
- Lightball (5-match line) → on detonation, fires one instant projectile at every enemy currently on screen. Mass single-target rather than mass AoE.

Combo bonus

Cascading matches grant a combo multiplier. Each cascade tick adds +0.25× damage to subsequent spells in that cascade chain. A 4-cascade match-3 chain therefore casts spells at 2× damage. Players who plan board states for cascades dramatically out-damage players who only see the next match.

3.5 Procedural Generation

Magus Match has no hand-designed levels (though we may seed the first 3 levels with templates for tutorial purposes). Every level is generated at runtime from a difficulty seed.

Difficulty curve

A run's difficulty is tracked by an integer counter that starts at 1 and increments by 1 each cleared level. Difficulty drives all generation parameters:

Difficulty	Journey: budget	Journey: land density	Trial: monster count	Trial: walk speed
1–3	20 moves	30% (easy)	3 basic kobolds	Slow
4–7	18 moves	25%	5 kobolds, 1 tall kobold	Slow
8–12	16 moves	22%	6 kobolds, 2 tall kobolds	Medium
13–18	15 moves	20%	7 kobolds, 3 tall kobolds, occasional mini-boss	Medium
19+	14 moves	18%	Mixed waves with mini-bosses; multi-wave levels	Fast

Journey level generation

17. Pick start cell (corner) and goal cell (opposite corner).
18. Compute a candidate path of cells from start to goal using A* with random walk cost (creates aesthetically interesting non-straight paths).
19. Mark every cell on this candidate path as guaranteed-buildable. The player CAN solve the level — the path exists.
20. Seed land tiles into roughly half the candidate-path cells, plus randomly into ~20% of off-path cells (to give the player options and red herrings).
21. Fill remaining cells with weighted-stratified standard tiles (no accidental match-3s at start, per §3.7 generation rules).
22. Run a solver to verify a path can be completed within the move budget under typical play; if not, add bonus land tiles to candidate-path cells until solvable.

Trial level generation

23. Determine wave size and composition from difficulty table.
24. For each monster, generate a spawn timing (uniform-random across the level's expected duration), a horizontal spawn position (one of 5 lanes), and an HP value scaled to difficulty.
25. Generate the standard tile board with no land tiles, weighted stratified, no accidental matches.

26. Verify clearable: simulate a player-of-average-skill (~1.5 matches per second, average 20% damage efficiency) — if they wouldn't clear the level before any monster reaches the mage, reduce HP by 10% and re-test.

Seeded generation

All procedural generation uses a deterministic seeded RNG. The default behavior seeds from a random number at run start (one unique experience per run). Determinism still matters for QA replay and bug reproduction — given the same seed, the generator must produce the identical level sequence.

3.6 Run-Level Meta

Lives

Every run starts with 3 lives. Failing a level (Journey or Trial) costs 1 life. Lives do not regenerate during a run. There is no continue, no rewarded-ad revive, no in-app purchase to extend a run. Three strikes and the run ends. This intentionally simple structure keeps run length self-limiting and discourages grinding.

Scoring

Score accumulates across all cleared levels in a run:

Source	Points	Notes
Cleared Journey level	1000 base	Plus 50 per move remaining
Cleared Trial level	1000 base	Plus 100 per match-per-second above 1.0 average
Combo match (4+ in chain)	100 each	Cascades count separately
Power-up created	200 each	Rocket / TNT / Lightball
Power-up combo (e.g., Rocket+TNT)	500	Stretch goal
Difficulty multiplier	$\times (1 + 0.1 \times \text{difficulty})$	Applied to base level-clear points

Leaderboard

A single persistent global Top 10 leaderboard tracks the highest-scoring runs ever recorded. It's shown on both the Title screen (as a recurring "Hall of Heroes" rotation) and the Game Over screen (with the just-completed run's rank highlighted if it qualifies).

Player names default to a procedurally generated heroic title ("Sir Brave-Heart of the Eastern Reach") unless the player edits it. This avoids any login or account requirement for MVP.

3.7 Board Specification

Board dimensions

The play board is an 8×8 grid. 64 cells total. Origin (0, 0) is the top-left cell; X increases right, Y increases down. This is the authoritative board size — every level, every layout, every power-up radius is tuned against this grid.

Why 8×8

- **Readable at portrait mobile sizes.** On a 1080×1920 target, each cell renders at ~128 px square with margins — large enough for finger input without a zoom gesture.
- **Symmetric.** Square grids produce more interesting power-up geometries (L-shape and T-shape matches resolve identically regardless of orientation) and simplify the solver.
- **Power-up scales well.** A TNT's 3×3 blast zone clears 9/64 cells (~14%) — enough to feel impactful, not enough to trivialize a level. A Rocket clears 8/64 (~12.5%). A Lightball averages 12–16 tiles cleared depending on distribution — the big one.
- **Matches Royal Match's canonical board.** Most Royal Match levels use an 8-wide interior with masking. Keeping parity helps players who cross over from the source game read our board instantly.

Cell coordinate system

Cells are indexed as (col, row) where $\text{col} \in [0, 7]$ and $\text{row} \in [0, 7]$. Row 0 is the top row of the board (tiles spawn from here). Row 7 is the bottom row. The board occupies the Match Board section (the bottom 1120 px of the 1920 px reference screen), centered horizontally and vertically within that section. World-space position of a cell center is:

$\text{worldX} = \text{boardOriginX} + \text{col} \times \text{CELL_SIZE} + \text{CELL_SIZE}/2$

$\text{worldY} = \text{boardOriginY} + \text{row} \times \text{CELL_SIZE} + \text{CELL_SIZE}/2$

Where $\text{CELL_SIZE} = 128$ px at 1080×1920 reference resolution.

Cell states

Each of the 64 cells holds a Cell object with the following fields:

Field	Type	Purpose
tile	Tile null	The active tile occupying this cell. Null during cascade gaps.
blocker	Blocker null	Static obstacle: locked brick, cursed, box, mail, etc. Blockers live on a separate layer from tiles and persist across swaps.

Field	Type	Purpose
modifier	Modifier null	Transient state: 'frozen' (chain-locked), 'spawned-this-turn' (for juice).
isVoid	boolean	True for masked cells — level layouts can carve non-rectangular boards out of the 8×8 grid. Void cells never receive tiles.

Masking non-rectangular layouts

Even though the underlying grid is 8×8, individual levels (especially higher-difficulty Journey levels) may declare cells as isVoid to create irregular silhouettes — a circular cave, a hexagonal arena, a scroll-shaped layout. This is critical for late-game variety. Example mask (1 = playable, 0 = void):

```

      0 0 1 1 1 1 0 0      0 1 1 1 1 1 1 0      1 1 1 1 1 1 1 1      1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1      1 1 1 1 1 1 1 1      0 1 1 1 1 1 1 0      0 0 1 1 1 1 0 0
```

3.8 Tiles & Data Model

Tile types

Magus Match uses four tile types, each mapped to a spell school (per §3.4) and color-coded for instant readability. In Trial levels each type casts the same instant single-target spell with a distinct visual; in Journey levels all four types behave identically (matches paint paths via the special LAND tile type below).

ID	Name	Color	Base sprite	Role / Spell school
FIRE	Flame	Red (#EB5757)	fire_tile.png	Fire school — instant single-target spell in Trial. In Journey: standard tile.
ICE	Frost	Blue (#2D9CDB)	ice_tile.png	Ice school — instant single-target spell in Trial. In Journey: standard tile.
LIGHTNING	Bolt	Gold (#F2C94C)	lightning_tile.png	Lightning school — instant single-target spell in Trial. In Journey: standard tile.
EARTH	Stone	Green (#27AE60)	earth_tile.png	Earth school — instant single-target spell in Trial. In Journey: standard tile.
LAND	Path	Brown earth (#8B6F47)	land_tile.png	Journey-only special tile. Matching 3+ converts cells to permanent walkable path. Does not appear in Trial levels.

Power-up tile types

ID	Name	Created by	On detonation
ROCKET_H	Horizontal Bolt	Match of 4 in a row	Clears entire row on the board. In Trial, fires a sequence of instant projectiles, one per enemy along the row's projectile lane.
ROCKET_V	Vertical Bolt	Match of 4 in a column	Clears entire column on the board. In Trial, fires a sequence of instant projectiles, one per enemy in the column's lane.
TNT	Magical Bomb	L-shape or T-shape match of 5	Clears 3×3 on board. In Trial, fires multiple instant projectiles spread across the wave, one per enemy hit.
LIGHTBALL	Mage's Orb	Straight match of 5	On tap-swap with any tile, clears all tiles of that color. In Trial, fires one instant

ID	Name	Created by	On detonation
			projectile at every enemy on screen. The signature panic-button move.

Power-ups are the most fun moment in the game. In Trial levels they're game-savers — clearing a wave that's about to overrun the mage by hitting many enemies in rapid succession. In Journey levels they're still useful (a Rocket might paint half a row of LAND tiles instantly if those cells contain LAND), but they don't directly progress the goal.

Tile state model (TypeScript interface)

```
interface Tile { id: string; // uuid, stable across cascade frames
type: TileType; // FIRE | ICE | LIGHTNING | EARTH | LAND | ROCKET_H |
ROCKET_V | TNT | LIGHTBALL col: number; // 0..7 row: number;
// 0..7 state: 'IDLE' | 'SWAPPING' | 'MATCHED' | 'FALLING' | 'DETONATING' |
'CONVERTED_TO_PATH'; spawnedAt: number; // ms timestamp, for juice
timing}interface Blocker { type: BlockerType; // LOCK | CURSED | BOX |
METAL_PLATE | CHAIN | EGG hp: number; // adjacent matches required to
clear position: { col: number; row: number };
```

Tile distribution at board generation

The initial board is NOT fully random. A random 8×8 fill produces accidental matches (cascades before the player swipes) and boards with dead regions. Use weighted stratified sampling:

27. Generate each cell in reading order (top-left to bottom-right).
28. For each cell, build a candidate list of all 4 tile types.
29. Remove any type that would form a match-3 with the two already-placed cells directly above or to the left.
30. Choose uniformly from remaining candidates. If the list is empty (rare), use the type that forms only a 2-match, not a 3-match.
31. After fill, run the match solver. If zero valid moves exist, reshuffle. If fewer than 3 valid moves exist, reshuffle. This guarantees a board with player agency from move 1.

3.9 Core Algorithms

This section specifies the algorithms an AI code generator should implement. Each is described with inputs, outputs, and edge cases.

3.9.1 Swap validation

Input: source cell (c1, r1), target cell (c2, r2).

Rules:

- Target must be orthogonally adjacent (Manhattan distance of exactly 1).
- Both cells must be non-void and contain a tile.
- Neither cell may contain a blocker of type LOCK, METAL_PLATE, or BOX (these block swapping through them).
- If either tile is a LIGHTBALL, the swap is always valid (Lightball activation).
- Otherwise, after simulating the swap, at least one match-3 must exist on the board, OR one of the two tiles is a power-up. If neither condition holds, swap is invalid and tiles snap back.

3.9.2 Match detection

After a swap or cascade, scan for matches:

32. Horizontal pass: for each row, find runs of ≥ 3 same-type tiles.
33. Vertical pass: for each column, find runs of ≥ 3 same-type tiles.
34. Merge overlapping runs into Match groups. A match of 5 in an L shape is one group, not two.
35. For each group, classify: 3 = basic, 4-straight = Rocket, 5-straight = Lightball, L/T 5+ = TNT.
36. Return list of Match objects, each with: tiles[], shape, spawnPowerUp (TileType or null), spawnCell ({col, row}) — the cell where the power-up will materialize, chosen as the cell the player's swap landed on, or the center-of-mass of the match if triggered by a cascade.

3.9.3 Cascade resolution

After matches resolve and tiles clear, gravity applies. This is per-column, top-to-bottom:

37. For each column, starting from row 7 upward, count empty non-void cells. Each tile above an empty cell falls by that number of rows.
38. Empty cells at the top of each column are refilled by new tiles spawning from above the grid (row -1) and animating down. Spawn distribution uses the same weighted sampling as board generation.
39. After all tiles have settled, re-run match detection. If new matches exist, repeat the cascade. Otherwise, return to IDLE.
40. Concurrent matching: the player may initiate a new swap during cascade frames. Queue the swap; apply it when all tiles involved are in IDLE state. Do NOT freeze input during cascades — this is the Royal Match "fastest switcher" feel.

3.9.4 Power-up detonation

Each power-up has a deterministic blast pattern on the board:

Power-up	Cells cleared	Visual FX
ROCKET_H	All 8 cells in the row at the rocket's row index	Horizontal rocket sprite streaks across the row in ~200ms; board shakes 4px.
ROCKET_V	All 8 cells in the column at the rocket's column index	Vertical rocket sprite streaks down/up through the column in ~220ms; board shakes 4px.
TNT	3×3 box centered on TNT cell (with grid clipping at edges)	Explosion sprite burst, radial shockwave ring, board shakes 8px.
LIGHTBALL	All tiles of the color the Lightball was swapped with	Full-board lightning flash overlay, 12px screen shake, audio ramp.

3.9.5 Power-up combinations (stretch)

Royal Match's signature feel comes partly from combining power-ups. For MVP, implement only the high-impact combos:

- **Rocket + Rocket** → Clears full row AND full column at the swap intersection. Two rocket sprite streaks perpendicular.
- **TNT + TNT** → 5×5 blast on board. Maximum screen shake, overlapping explosion sprites, brief slow-motion.
- **Lightball + Rocket** → Every tile of the chosen color converts into a Rocket and fires simultaneously. Screen fills with streaks.
- **Lightball + Lightball** → Clears the entire board. If the level objective is already partially met, this usually wins the level outright. Big victory flourish.

3.10 Game State Machine

The game runs on a single finite state machine with the following states. Only one state is active at a time. Transitions are triggered by user input, timers, or algorithmic completion.

State	Enters when	Input?	Exits to
BOOT	Game loads	No	→ TITLE (after assets load)
TITLE	Game finished booting	Tap Play	→ LEVEL_INTRO
LEVEL_INTRO	Level begins (Journey or Trial)	No — 1.5s scene establishment, mage strikes pose, prince is shown cowering	→ IDLE
IDLE	Board is settled, waiting for input	Yes — swipe to swap	→ SWAP_VALIDATING on swipe
SWAP_VALIDATING	Player initiated swap	No — 200ms swap animation + check	→ RESOLVE_MATCHES (valid) or IDLE (invalid snap-back)
RESOLVE_MATCHES	Matches detected	Yes (queued) — match/cascade animations	→ POWERUP_DETONATE if created, else STAGE_UPDATE
POWERUP_DETONATE	Power-up triggered	Yes (queued)	→ RESOLVE_MATCHES (detonation often creates more matches)
STAGE_UPDATE	Board changes have settled	No — apply spell damage (Trial) or advance mage one step (Journey); ~400ms	→ CHECK_WIN
CHECK_WIN	Stage has updated	No	→ IDLE (continue), WIN (level cleared), or LOSE (level failed)
WIN	Journey: mage at goal. Trial: all monsters defeated.	No — 3.5s victory cinematic ending in cage-yank	→ INTERSTITIAL
LOSE	Journey: out of moves. Trial: monster reached mage.	No — 1.5s fail beat	→ INTERSTITIAL or GAME_OVER if 0 lives left
INTERSTITIAL	Between levels in a run	No — "Level N" stinger, ~1s	→ LEVEL_INTRO (next level)

State	Enters when	Input?	Exits to
GAME_OVER	All 3 lives lost	Tap Try Again / View Leaderboard	→ TITLE (replay) or stays for leaderboard browse

Input queueing

During RESOLVE_MATCHES and POWERUP_DETONATE, the player CAN swipe. Their swipe is captured and applied the moment the state machine returns to IDLE. This queue has a maximum depth of 1 — only the most recent swipe is retained. Older queued swipes are discarded. This matches Royal Match's "concurrent matching" behavior.

3.11 Hero Stage Renderer & Character Animation

The Hero Stage is the top 600 px of the 1920 px reference screen — a 3D scene that houses every animated character: the mage-knight protagonist, the prince in his cage, the monsters during Trial levels, and any scenario-specific scene elements. Character animation uses 2D cutout puppets rigged in Blender — body parts as image planes, alpha-trimmed, weight-painted, and posed via a bone armature — composited into the 3D stage. This is preferable to per-frame sprite-sheet animation for its expressiveness and asset-volume efficiency, and the 3D framing supports the eventual Horizon Worlds port where the player can occupy the stage in headset mode.

Why Blender cutout puppets

- **Expressive character motion.** Skeletal-rigged 2D cutout characters can produce a wider variety of poses and reactions per character than fixed-frame sprite sheets, with no per-frame redraw cost.
- **Animation reuse.** One mage-knight rig animates idle, casting, victory, and walking — the same skeleton drives all four animations. We don't need to redraw the full character per frame.
- **AI-art friendly.** Each body part (head, torso, sword arm, staff arm, legs) is a separately AI-generated PNG, layered on Blender image planes. The AI generates ONCE per part; Blender handles motion. This is dramatically more efficient than generating 60 frames of consistent character.
- **Exportable to engine.** Blender bakes character animations to a sprite-sheet-like atlas + skeletal data, exportable to Phaser as Spine-style runtime animation, or to Horizon as a sequence of skinned 2D meshes.

Pipeline reference

Xan owns the rigging stage of this pipeline. The reference workflow is documented at theartsquirrel.com/4283/2d-cutout-animation-in-blender (Krita layer prep → Blender Image-as-Plane import → Tessellate Texture Plane add-on for alpha trim → bone armature → weight paint → animation timeline → atlas export). The team should standardize on this exact workflow for all five MVP characters.

Required character rigs (MVP)

Character	Animations needed (MVP italicized)	Estimated rigging complexity
Mage-knight protagonist	Idle, Cast Spell, Victory, Walk Cycle, Cast Spell (Combat Variation)	High — humanoid with two distinct cast poses
Magical Spell projectile	Cast on Tile, Moving Projectile, Collision with Target	Low — 1 sprite + simple position/rotation tweens
Basic Monster (kobold)	Walk Cycle, Taking Damage, Defeat	Medium — biped, asymmetric proportions

Character	Animations needed (MVP italicized)	Estimated rigging complexity
Prince in Cage	Cowering in Cage, Victory	Low — confined to cage frame, mostly facial+arm motion
Stronger Monster (tall kobold)	Walk Cycle, Taking Damage, Defeat	Medium — re-uses kobold rig with proportional scale and added detail

HeroStage data structure

```
interface HeroStage { levelType: 'JOURNEY' | 'TRIAL'; mage: MageActor;
// the protagonist puppet prince: PrinceActor; // the captive
monsters: MonsterActor[]; // populated in Trial; empty in Journey
pathRenderer: PathRenderer | null; // populated in Journey; null in Trial backdrop:
BackdropAsset; // current biome/scene art}interface MageActor { position:
{ x: number; y: number }; currentAnim: 'idle' | 'cast' | 'cast_combat' | 'walk' |
'victory'; facing: 'left' | 'right'; rig: BlenderRig; // imported
from .json + atlas}interface MonsterActor { type: 'kobold' | 'tall_kobold' |
'mini_boss'; hp: number; maxHp: number; position: { x: number; y: number };
walkSpeed: number; // px/sec, downward; constant per spawn
currentAnim: 'walk' | 'damage' | 'defeat'; rig: BlenderRig;}
```

Frame-update rule

Every render frame, the Hero Stage:

41. Updates the mage's current animation based on game state (casting after match, idle when board is settled, victory when level cleared, walking in Journey when path advances).
42. Updates monster positions if in Trial: $\text{walkSpeed} \times \text{deltaTime}$ moves each monster downward; clamped to mage's standing line which triggers fail.
43. Updates prince animation: COWER during gameplay, VICTORY on level clear.
44. Renders the backdrop, then mage, then prince/cage, then monsters and projectiles in z-order.

The victory cinematic

On level clear, the Hero Stage runs a scripted sequence:

45. Mage plays VICTORY animation (~1.5s).
46. Camera pushes slightly toward the prince's cage.
47. Prince plays VICTORY animation: cheers, fist-pump, jumping in place (~1s).
48. Sound effect: "WHOOSH." Cage is yanked upward (off-screen) at high speed; one frame of motion blur sells the snatch.
49. Mage plays a stunned reaction (1 second of head-scratching).
50. Cut to next level transition. Total cinematic length: ~3.5 seconds — slightly longer than fail beats so wins feel earned but not draggy.

3.12 Level Data — Generation Schema

Magus Match levels are procedurally generated at runtime. Levels are NOT serialized JSON files — they are produced by the generator (§3.5) from a seed + difficulty pair. This section specifies the runtime data structures the generator produces.

```
interface GeneratedLevel {  type: 'JOURNEY' | 'TRIAL';  difficulty: number;
// 1..N, current run progress  seed: number;                      // RNG seed —
// randomized per run, captured for QA replay  initialBoard: Cell[][];          // 8x8,
// fully populated on entry  // Journey-specific  journey?: {    moveBudget: number;
startCell: { col: number; row: number };    goalCell: { col: number; row: number };
landTilePositions: { col: number; row: number }[]; // pre-seeded land tiles
candidatePathSolution: { col: number; row: number }[]; // for solver verification only
}; // Trial-specific trial?: {    waveManifest: MonsterSpawn[];    expectedDuration:
number;          // seconds, for solver verification  };}interface MonsterSpawn {  type:
'kobold' | 'tall_kobold' | 'mini_boss';  spawnTimeMs: number;          // ms after
level start  lane: number;          // 0..4, horizontal spawn position  hp:
number;          // pre-scaled by difficulty}
```

Difficulty progression in a typical run

Cleared levels	Difficulty	Typical experience
0 (level 1)	1	Tutorial-feel Journey level. 20 moves, sparse land, easy win.
1–3	2–4	Alternating Journey/Trial. Player learns both modes. Most players survive this stretch with 3 lives intact.
4–7	5–8	Difficulty noticeable. Trial waves get tall kobolds. Average player loses first life around here.
8–12	9–13	Skill check zone. Average player ends run between levels 8 and 12.
13–18	14–19	Skilled player territory. Mini-bosses appear. High-score runs.
19+	20+	Endgame for committed players. Multi-wave Trials, tight Journey budgets. Designed to be hard.

3.13 Tuning Constants

A single tuning.ts file centralizes the numerical feel parameters. Every value here is subject to change during playtesting, but these are reasonable starting points based on Royal Match's observed feel:

Constant	Value	Effect if increased
SWAP_DURATION_MS	150	Feels heavier, less snappy
MATCH_POP_DURATION_MS	220	Matches feel more celebratory but the game slows
CASCADE_FALL_DURATION_PER_ROW_MS	60	Cascades become majestic but less responsive
CASCADE_ROW_STAGGER_MS	20	Tiles fall more clearly in sequence vs. all-at-once
INPUT_QUEUE_MAX_DEPTH	1	Players can queue more — feels faster but loses precision
CAMERA_SHAKE_MIN	2 px	Even small matches shake — can become fatiguing
CAMERA_SHAKE_MAX	12 px	TNT feels more powerful but can cause nausea in headset port
MAGE_WALK_DURATION_MS	350	Per-cell walk animation in Journey — slower feels weightier
MONSTER_WALK_SPEED_PX_S	30 (base)	Trial monster downward speed at difficulty 1; scales with difficulty
SPELL_PROJECTILE_VISUAL_MS	80	Visual streak duration. Damage applies on frame 0; this is just feedback. Lower = snappier, higher = more readable trail.
FIRST_MATCH_HINT_DELAY_MS	2500	If player hasn't moved after this long on level 1 of a run, glow a suggested match

Difficulty curve — target survival rates

Magus Match's procedural generator should be tuned so that an average player progresses according to these survival rates per level. These targets directly drive the parameters in §3.5:

- Level 1: 95%+ survival (no lives lost)
- Level 2–4: 88–92% per level
- Level 5–8: 75–85% per level (first life typically lost in this range)
- Level 9–12: 60–70% per level
- Level 13+: 50% per level, asymptoting toward ~40% at level 20+

Average run length under these targets: ~8–10 cleared levels per run. Top decile of players: ~15+ cleared levels. If live metrics deviate from these bands by more than 15 points across a 1000-run sample, the

difficulty-scaling formula in §3.5 should be retuned (move budget for Journey, monster HP for Trial, in that priority order).

3.14 Implementation File Layout

Suggested Phaser 3 + TypeScript project structure. Codex and Cursor should scaffold against this layout:

```
src/ main.ts // Phaser game bootstrap scenes/ BootScene.ts
// asset loading TitleScene.ts // logo + Play, leaderboard, attract
loop JourneyScene.ts // Journey level orchestrator TrialScene.ts
// Trial level orchestrator InterstitialScene.ts // 'prince yanked'
transition between levels GameOverScene.ts // run end, leaderboard, Try
Again board/ Board.ts // 8x8 grid, cell state, masking
Tile.ts // tile sprite + state machine LandTile.ts
// Journey-specific persistent path tile MatchSolver.ts // match
detection (section 3.9.2) Cascade.ts // gravity and refill (section
3.9.3) PowerUps.ts // power-up creation and detonation heroStage/
HeroStage.ts // top-30% manager MageActor.ts //
protagonist puppet driver PrinceActor.ts // captive puppet driver
MonsterActor.ts // monster puppet driver (Trial) SpellProjectile.ts
// instant streak + impact VFX PathRenderer.ts // Journey path
visualization BlenderRig.ts // skeletal anim runtime, atlas loader
rigs/ // exported .json + atlas per character generator/
LevelGenerator.ts // top-level dispatcher (Journey or Trial)
JourneyGenerator.ts // path-feasible board generator TrialGenerator.ts
// wave manifest generator DifficultyTable.ts // scaling parameters from
section 3.5 SeededRng.ts // deterministic RNG; seed captured per run
for QA replay meta/ Run.ts // current run state: lives, score,
difficulty Scoring.ts // points calculation Leaderboard.ts
// local + remote storage state/ GameStateMachine.ts // the FSM from
section 3.10 InputQueue.ts // queued-swipe handling data/
tuning.ts // section 3.13 constants audio/ AudioController.ts
// match pings, spell stings, ambient beds, victory cinematic telemetry/
Metrics.ts // section 7.3 event logging
```

4. Game Feel & Aesthetics

4.1 Visual Direction

Magus Match's aesthetic sits between hand-painted storybook fantasy (think "Hollow Knight" 2D moods or Owlboy backgrounds) and the cozier, brighter palette of mainstream casual match-3 like Royal Match or Empires & Puzzles. Backgrounds are AI-generated 2D illustrations with a unified prompt language: warm sunlit forests, ruined stone paths, dusty crypts, glowing crystal caverns. Tiles are crisp readable icons in a saturated, slightly stylized fantasy alphabet (a flame, a snowflake, a lightning bolt, a stone shard). Characters are Blender-rigged 2D cutouts that allow more expressive motion than fixed frames.

Color palette

- **Royal Purple** (#4B2E83) — backgrounds, UI chrome, brand accent
- **Aged Gold** (#C8A24B) — accents, victory glow, mage's magical sigils
- **Parchment Cream** (#F5E9C9) — UI panels, modals, leaderboard frames
- **Spell School Colors** — Fire Red, Ice Blue, Lightning Gold, Earth Green — each high-saturation and perceptually distinct at thumbnail size, AND mapped 1:1 to the four tile types so players intuit "red tile = fire spell" instantly

Character design

- **Mage-knight protagonist** — A heroic young figure in flowing cape, simple pauldron, staff in one hand, sword on hip. Anonymous heroic archetype rather than a defined personality. Friendly but capable. Designed to read at thumbnail size: silhouette is the staff + cape + crown-like helm.
- **Prince in cage** — Younger character, finer clothes, comically resigned expression most of the time. Expressive enough to sell the running gag — eyes track the audience, then go wide as the cage gets snatched. The prince does most of the comedic work in this game.
- **Kobolds and tall kobolds** — Cartoonish, not menacing. Goofy underbites, oversized weapons, slightly bow-legged walks. Easy to hit, easy to defeat — these are the punching bags of the game.
- **The Unseen Abductor** — Never visible. Hinted only by the giant hand or rope or claw that appears just at frame edges to snatch the cage. Different visual hint each level: a giant glove, a tentacle, a magical hand, a hook. The mystery is the meta-progression.

4.2 Animation & Juice

This is a juice-first game. Every interaction has exaggerated feedback:

- Tile swap: 80ms snappy lerp with 10% overshoot
- Match clear: pop + sparkle + magical chime, school-colored particle burst
- Spell projectile launch: tile-cell flashes white, projectile streaks upward as a fast straight line with a trailing particle stream

- Spell impact (Trial): monster flinches, damage popup, school-themed VFX (flame burst / ice shatter / electric arc / dust burst)
- Path advance (Journey): mage walks one cell with a confident step animation, leaving a soft magical glow trail
- Power-up creation: 300ms slow-motion pulse highlighting the new power-up, with a deeper magical chime
- Victory cinematic: 1.2s slow-motion on the prince celebrating, gold halo, bells; then sudden snap to high-speed cage yank
- The cage yank: ONE frame of motion blur. The whole snatch is over in 250ms. The comedic timing depends on this snap being unexpected and instant.

4.3 Audio

Audio borrows from the Sonnis GDC bundles already in the team's library. Key cues:

- Tile match: bright xylophone ping, pitch-shifted up per cascade step in a combo
- Spell projectile: school-themed whoosh (crackling fire / icy chime / electric crackle / earthy thud)
- Spell impact: school-themed thud (fiery boom / icy shatter / sharp electric snap / heavy stone thunk)
- Path-tile conversion (Journey): a satisfying "thunk" of stone setting
- Mage walk: soft footsteps with subtle magical chime
- Monster damage: comedic "oof" with rising pitch, monster-defeat: descending three-note 'wah-wah-WAH'
- Victory sting: 1.2s triumphant brass+bell flourish leading INTO the cage-yank "WHOOSH"
- The WHOOSH itself: a short, sharp pneumatic-tube swoosh — instantly recognizable, the audio trademark of the running gag
- Run-end (3 lives lost): a sympathetic but hopeful descending fanfare; not punishing

5. Scope & MVP

5.1 MVP Screens (Per Brief's 3-Screen Limit)

#	Screen	Purpose
1	Title / Attract Screen	Logo, Play button, leaderboard preview. Auto-playing demo loop. Doubles as the Reels creative.
2	Play Screen	The entire game. Three sections: Hero Stage (top 600 px), Gameplay UI strip (200 px), Match Board (1120 px). One layout for both Journey and Trial — content varies.
3	Game Over Screen	End of run. Final score, high score, leaderboard top 10, Try Again button. Built for screenshot-share.

No map screen. No castle-decoration meta. No shop. No settings menu in MVP — mute toggle lives as a tiny icon on the Play screen.

5.2 Content Scope for Launch

- **Level types:** 2 (Journey + Trial), procedurally generated. No hand-authored levels in MVP.
- **Tile types:** 4 spell schools (Fire, Ice, Lightning, Earth) + 1 special (Land)
- **Power-ups:** 4 (Horizontal Bolt, Vertical Bolt, Magical Bomb, Mage's Orb)
- **Characters with Blender rigs:** 5 (Mage-knight, Prince, Magical Spell, Basic Kobold, Tall Kobold)
- **Animations across all characters:** MVP set per the table in §3.11 (highlighted items in the source brief)
- **Backdrops:** 3 biome variants (Forest, Crypt, Crystal Cave) rotated by difficulty band — AI-generated, no character animation, just parallax

5.3 Explicit Out-of-Scope

Per the brief's scope guardrails, the following are explicitly out of scope for Magus Match and should be deferred or dropped entirely:

- Castle decoration meta or any other persistent meta-progression
- Coins / currency / shop / in-app purchases
- Energy regeneration system (the 3-lives-per-run is the only life mechanic)
- Teams / alliances / social features beyond leaderboards
- Tournaments / events / battle pass
- Tutorial beyond the pre-highlighted first match in level 1
- Save/resume mid-run (a level is short enough that restarting is fine; if a run is interrupted it's lost)
- Account / login system — local high scores only for MVP, with optional name entry

- Boss-only levels — mini-bosses appear within standard Trial waves but no dedicated boss level type
- Hand-designed levels (procedural generation only)

6. Technical Direction

6.1 Prototype Stack

Following the team's established toolchain for Batch 2:

- **Engine:** Phaser 3 + TypeScript for the prototype, consistent with Slipstream and the other Batch 2 games.
- **Physics:** None required. The match-3 uses Phaser's built-in tween system for cascades. Spell projectiles are instant — a fast streak visual + immediate damage application; no rigid bodies, no collisions, no flight-time gating.
- **Character animation:** 2D cutout puppets rigged in Blender per Xan's pipeline (theartsquirrel.com/4283). Each character is exported as a sprite atlas + skeletal animation JSON, loaded at runtime via a lightweight runtime (Spine-style or custom).
- **Asset pipeline:** Backdrops, tiles, and character body parts are AI-generated 2D images. Body parts are imported as Blender Image Mesh Planes, alpha-trimmed via the Tessellate Texture Plane add-on, rigged with a bone armature, weight-painted, and animated. Animations are baked to atlas + JSON for runtime playback.
- **Build target:** Vertical 1080×1920, scaled to fit. Touch input only.

6.2 Horizon Worlds Port

Per the brief, the Phaser prototype is a proving ground; the shipping version runs on Meta Horizon Worlds. Key considerations for the port:

- Match-3 logic, generator, and run meta are pure TypeScript — port directly to Horizon's TypeScript API.
- 2D tile rendering on Horizon: use a flat quad stage facing the player, with UI panels parented to the camera.
- No physics engine required on Horizon. The entire game is 2D sprites, tweened transforms, and skeletal-animated character puppets — Horizon handles all of this natively.
- Blender rig export needs a runtime that's compatible with Horizon's scripting environment. Spine has Horizon support; if our custom runtime ships first, validate it on Horizon early in week 2.
- Touch input on mobile Horizon → map to Horizon's pointer event model. Desktop/headset → raycast from controller.
- Audio: Horizon's spatial audio system, with 2D-positioned cues on the UI plane.
- Leaderboards: Horizon's persistent player data API can store per-player high scores; for shared leaderboards, validate Horizon's networking model in week 2.

6.3 Performance Budget

Metric	Target	Note
Frame rate (mobile Horizon)	60 fps sustained	Drops to 45 acceptable during multi-monster Trial waves with 5+ active spell projectiles only
Active rigged puppets	≤ 8 simultaneously	Mage + prince + up to 6 monsters in a peak Trial wave
Character atlas size per rig	$\leq 2048 \times 2048$	Single atlas per character, holds all body parts and animation frames
Tile board size	8 cols $\times$ 8 rows	64 tiles — see §3.7
Asset memory total	< 100 MB	Atlas-packed, compressed textures across characters + backdrops + tiles
Load time to first playable frame	< 2s	Critical for Reels conversion — any longer and the install drops off

7. Distribution Strategy — Designing for Reels

The brief is explicit: these games are promoted and distributed via Instagram Reels, and the goal is clicks and plays to study metrics and user behaviors. Magus Match is designed to be Reels-native, with the running-gag cage-yank as the central marketing asset.

7.1 The 15-Second Reel

Structure of the primary marketing creative:

- 51.0:00–0:03 — Cold open on the mage, mid-battle. Spell projectiles flying. Tension.
- 52.0:03–0:08 — Quick cuts of gameplay: matches firing spells in Trial, painting paths in Journey. The two modes get equal screen time.
- 53.0:08–0:12 — Victory cinematic: prince celebrates, then the WHOOSH cage-yank. The visual punchline.
- 54.0:12–0:15 — Install CTA. Caption: "Save the prince. Again. And again."

The cage-yank is the hook — it's the moment that turns confused viewers into installers, because it's the moment that explains why the game is endless.

7.2 What Makes This Game Screenshot-Worthy

- The cage-yank gag is a 3-second loop perfect for Reels — endlessly repeatable across the run, always different in micro-context (different backdrop, different abductor visual).
- The mage-knight and prince's expressions are designed to be recognizable at 100×100 thumbnail size; the running gag works in tiny preview frames.
- Two distinct level types means we can cut Trial-focused and Journey-focused ad variants from the same game — different mechanical hooks for different audience segments.
- Color palette is high-contrast against Instagram's white/grey chrome — the purple+gold brand pops in feed.

7.3 Metrics to Instrument

Since the stated goal is studying metrics and user behaviors, Magus Match should log the following from day one of its Reels launch:

- Time to first match (target: < 6s from game load)
- Survival rate per level number across runs (per the §3.13 difficulty curve targets)
- Average run length (target: 8–10 cleared levels per run for an average player)
- Journey vs Trial preference: which level type ends runs more often, which type produces more replays
- Try Again click-through rate within 3s of run end (target: > 50%)

- Share button engagement rate on Game Over screen (the primary viral-loop signal)
- Session length and runs-per-session distribution

8. Risks & Open Questions

8.1 Known Risks

Risk	Severity	Mitigation
Two level types doubles the design + tuning surface vs. a one-mode game	High	Build Journey to playable first (it's simpler and includes the tutorial-first-level slot). Trial second. Don't tune them in parallel — finish Journey before starting Trial implementation.
Blender cutout rig pipeline is novel for the team; rigging time is unpredictable	High	Xan to rig the mage-knight FIRST as a reference and time-box check. If rigging the mage takes more than 3 days, descope tall kobold + mini-boss to a single basic-kobold variant.
AI-generated character body parts have inconsistent style across the 5 characters	Medium	Lock a single AI prompt template + style reference image; generate all body parts for all 5 characters in one batch from the same seed family. Re-roll any inconsistencies BEFORE rigging starts.
Procedural Trial levels feel grindy when seeds repeat similar wave patterns	Medium	Use the difficulty band to gate which monster compositions can roll; ensure each band has at least 4 distinct wave templates that the seed selects from.
Players don't understand Journey mechanics on first encounter	Medium	Level 1 is GUARANTEED to be a Journey. Pre-highlight a land-tile match at the start. Show the mage's start cell and goal cell with arrows for the first 3 seconds.
Run length feels too long or too short relative to snackable scope	Low-Medium	5-minute upper bound from brief is comfortable for ~10 cleared levels. If average exceeds this, tighten the lives mechanic (cap at 2 lives instead of 3) or accelerate difficulty scaling.

8.2 Open Questions for Team Review

- **Shared mascot?** Batch 1 used Ember as a unifying character. Does Magus Match feature Ember in a cameo role (e.g., Ember as the mage's familiar, riding on the staff), or does the mage stand alone?
- **Reveal the abductor?** The running gag depends on the abductor staying mysterious. Do we EVER show them — at run end? Tease them across multiple runs? Or do they remain unseen forever, with the mystery itself being the meta-progression?
- **Level type weights?** 50/50 Journey/Trial after level 1 is the default. Should the weights shift over a run (more Trials at higher difficulty)? Or should the player be allowed to choose between two level types each transition for some agency?

- **Combo bonuses across level types?** Should clearing 3 Journey levels in a row grant a starting power-up on the next level? Or is the no-meta principle worth defending?
- **Horizon-specific features?** Does the Horizon port lean into the platform — e.g., the Hero Stage rendered as a 3D scene the camera is inside, so in headset mode you're RIGHT THERE with the mage as the cage gets snatched — or does it stay a flat 2D port?

9. Next Steps

Week 1

- Lock GDD in batch review meeting
- Xan: rig mage-knight character as pipeline proof; time-box at 3 days
- Generate first-pass AI art for all 5 characters' body parts (single batch, locked style)
- Build Phaser 3 match-3 skeleton + 8×8 board (Codex-generated from PLAN.md)
- Build LevelGenerator + JourneyGenerator with seeded RNG; produce a few sample levels

Week 2

- Mage-knight rig integrated into Phaser via runtime; idle + cast + walk animations playing
- Journey level type playable end-to-end (level 1 only; no Trial yet)
- First Horizon smoke test — character rig animation + match-3 board at 60fps
- First juice pass: tile animations, match VFX, mage walk feel

Week 3

- Trial level type playable end-to-end with kobold spawning, spell projectiles, damage
- Run-meta layer: lives, scoring, level transitions, victory cinematic with cage-yank
- Tall kobold + mini-boss rigs animated in Blender, integrated
- Game Over screen + local leaderboard

Week 4 (ship window)

- Full port to Horizon Worlds
- Reels creative cut from in-engine footage — multiple cage-yank clips
- QA pass on Horizon build
- Instagram Reels soft launch
- Metrics instrumentation live; begin data collection

— End of Document —